

Agenda

- Java Doc
- UML Diagrams
- GUI in Java
- MVC Pattern
- JUnit

Comments and embedded documentation

- There are two types of comments in Java
 - First type
 - » Is the traditional C style comment
 - » Begin with a `/*` and continue, possibly across many lines, until a `*/`
 - » Example: `/* This is a comment * that continues * across lines*/`
 - Second type
 - » Comes from C++
 - » It is the single-line comment
 - » Starts at a `//` and continues until the end of the line
 - » Example: `// This is an one-line comment`

Comment documentation

- Link the code to the documentation by:
 - Putting everything in the same file
 - Using
 - » A special comment syntax to mark the documentation
 - » A tool to extract those comments and put them in a useful form
 - ◆ The tool to extract the comments is called *javadoc*, and it is part of the JDK installation

Overview of the basics of **javadoc**

- All of the **javadoc** commands occur only within ***/*XXX*/*** comments
- **Standalone doc tags**
 - Start with a ‘@’ and are placed at the beginning of a comment line
- **@see**
 - Allow to refer to the documentation in other classes
 - Javadoc will generate HTML with the **@see** tags hyperlinked to the other documentation

@see *classname*
- **{@docRoot}**
 - Produces the relative path to the documentation root directory

Overview of the basics of *javadoc* - Some example tags

- **@version**

- Any significant information you want to include

@version *version-information*

- **@author**

- Information about author

@author *author-information*

» *author-information*:

- ◆ *Your name*
- ◆ *Email address* or any other appropriate information

- You can have multiple **author** tags for a list of authors, but they must be placed consecutively

Overview of the basics of *javadoc* - Some example tags

- **@since**
 - Indicate the version of this code that began using a particular feature
- **@param**
 - Used for method documentation
 - @param** *parameter-name* *description*
 - » *parameter-name* - is the identifier in the method parameter list
 - » *description* - is text description of the parameter
 - ◆ The description is considered finished when a new tag is encountered
 - You can have any number of these, presumably one for each parameter

Overview of the basics of *javadoc* - Some example tags

- **@return**

- Is used for method documentation

@return *description*

» *description* describes the meaning of the return value

- **@throws**

- Is the exception tag

@throws *fully-qualified-class-name description*

» *fully-qualified-class-name* gives an unambiguous name of an exception class that's defined somewhere

» *description* tells you why this particular type of exception can emerge from the method call

- **@deprecated**

- Is used to indicate a features that no longer can be used, since sometime in the future it is likely to be removed

Documentation example

```
//: c02:HelloDate.java
```

```
import java.util.*;
```

```
/** The first Thinking in Java example program.
```

```
 * Displays a string and today's date.
```

```
 * @author Bruce Eckel
```

```
 * @author www.BruceEckel.com
```

```
*/
```

```
public class HelloDate {
```

```
/** Sole entry point to class & application
```

```
 * @param args array of string arguments
```

```
 * @return No return value
```

```
 * @exception exceptions No exceptions thrown
```

```
*/
```

```
public static void main(String[] args) {
```

```
    System.out.println("Hello, it's: ");
```

```
    System.out.println(new Date());
```

```
}
```

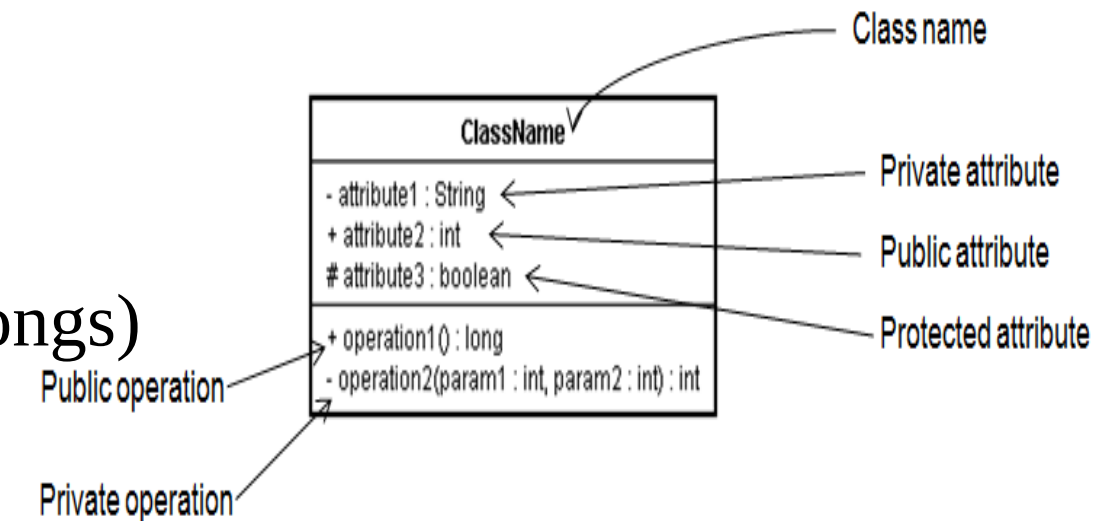
```
}
```

UML diagrams

- Class Diagrams
 - Is static diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects
- Use case diagrams

UML diagrams – Class Diagram

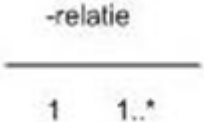
- **Notation and representation:** A class is graphically represented as a rectangle (see Figure 1) in which the following information is specified:
- *Class name*
 - Used to distinguish the class from other classes
 - Simple or qualified (prefixed by the name of the package to which the class belongs)
- *Class attribute*
 - Named property describing a range of values that instances of the property may hold
 - Shared by all objects of that class
- *Class operation*
 - Implementation of a service that can be requested from any object of the class to affect behavior
 - Can be specified by stating its signature



UML diagrams – Class Diagram

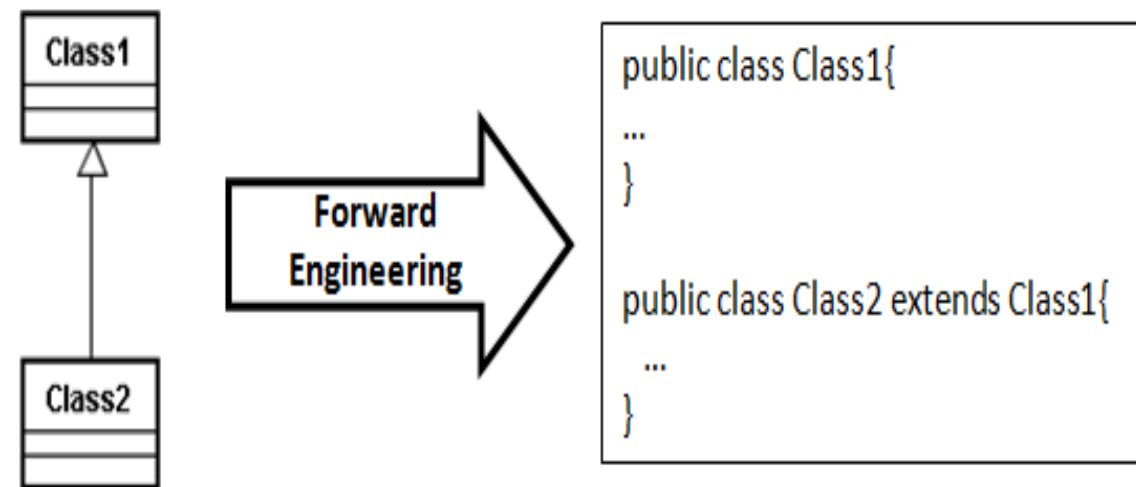
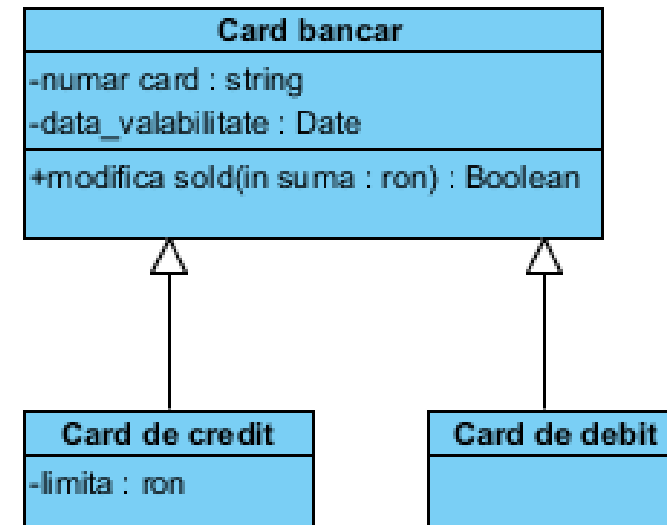
- **Relationships** - A relationship models the way things can connect to one another, either logically or physically
- **Notation and Representation:** A relationship is represented as a path with different kinds of lines used to distinguish the kinds of relationships
- **Types:** Dependencies, Associations, Generalizations, Realizations

UML diagrams – Class Diagram

Type of relationships	Definition	Representation
Inheritance	An inheritance relationships between two classes indicates that a child class inheritances the features of the parent class. The child has attributes and operations in addition to those found in its parents.	
Association	An association relationship specifies that objects of one thing are connected to objects of another thing. An association can have a <i>name</i> used to describe the nature of the relationship. When a class participates in an association, it has a specific <i>role</i> that it plays in that relationship. The <i>multiplicity</i> of an association's role represents a range of integers specifying the possible size of the set of related objects	
Dependency	A dependency relationship between two things states that one thing uses the information and services of the other thing, but not necessarily the reverse. In the case of classes, the dependency relationship is used to show that one class uses operations from another class or it uses variables or arguments typed by the other class => if the used class changes, the operation of the other class may be affected as well.	
Aggregation	Aggregation is a special kind of association used to model a „whole/part” relationship, in which one class represents a larger thing („the whole”), which consists of smaller things (the „parts”). Aggregation indicates that a parent class has elements of the child class type. If we associate the verb "to have" to express the connection between the two classes, in the aggregate we use the verb "contain".	 
Composition	Composition is a variation of simple aggregation in which the child class exists only if an instance of the parent class exists.	
Realization	A realization relationship is a semantic relationship between classifiers (e.g classes, interfaces, collaborations use cases) in which one classifier specifies a contract that another classifier guarantees to carry out	

UML diagrams – Class Diagram

- Example of inheritance:



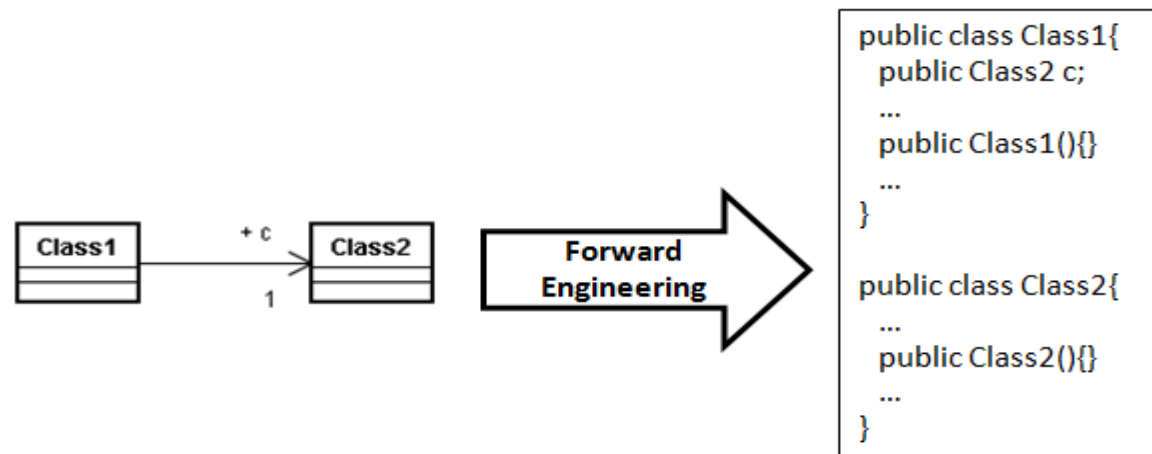
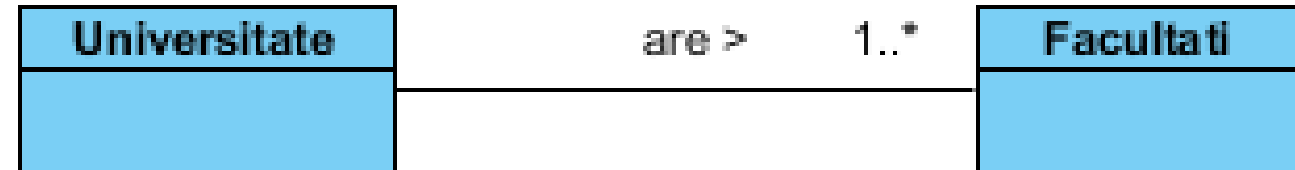
Forward engineering for the generalization relationship

UML diagrams – Class Diagram

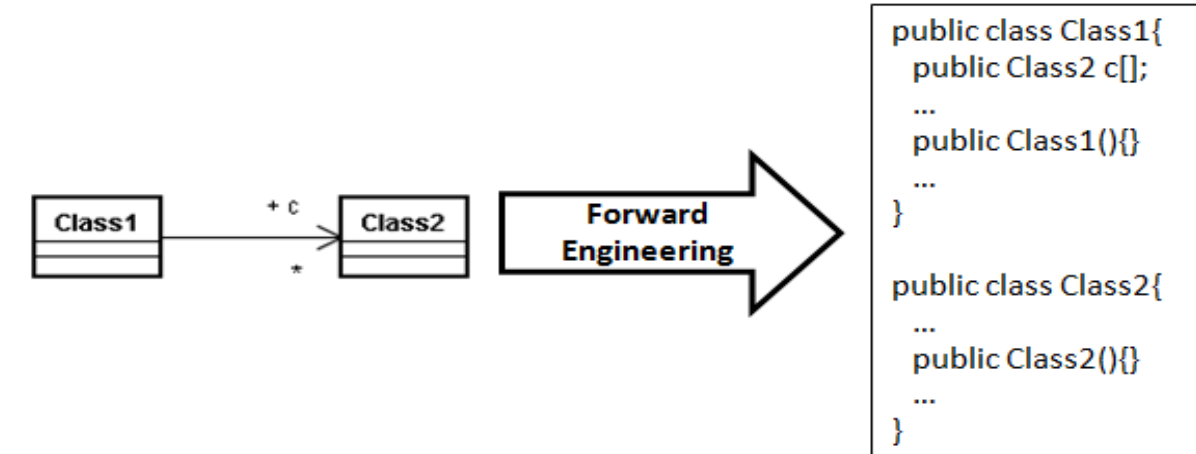
- Example of association
 - A university is administrated by a single rector and a rector administrate a single university



- A university has several faculties



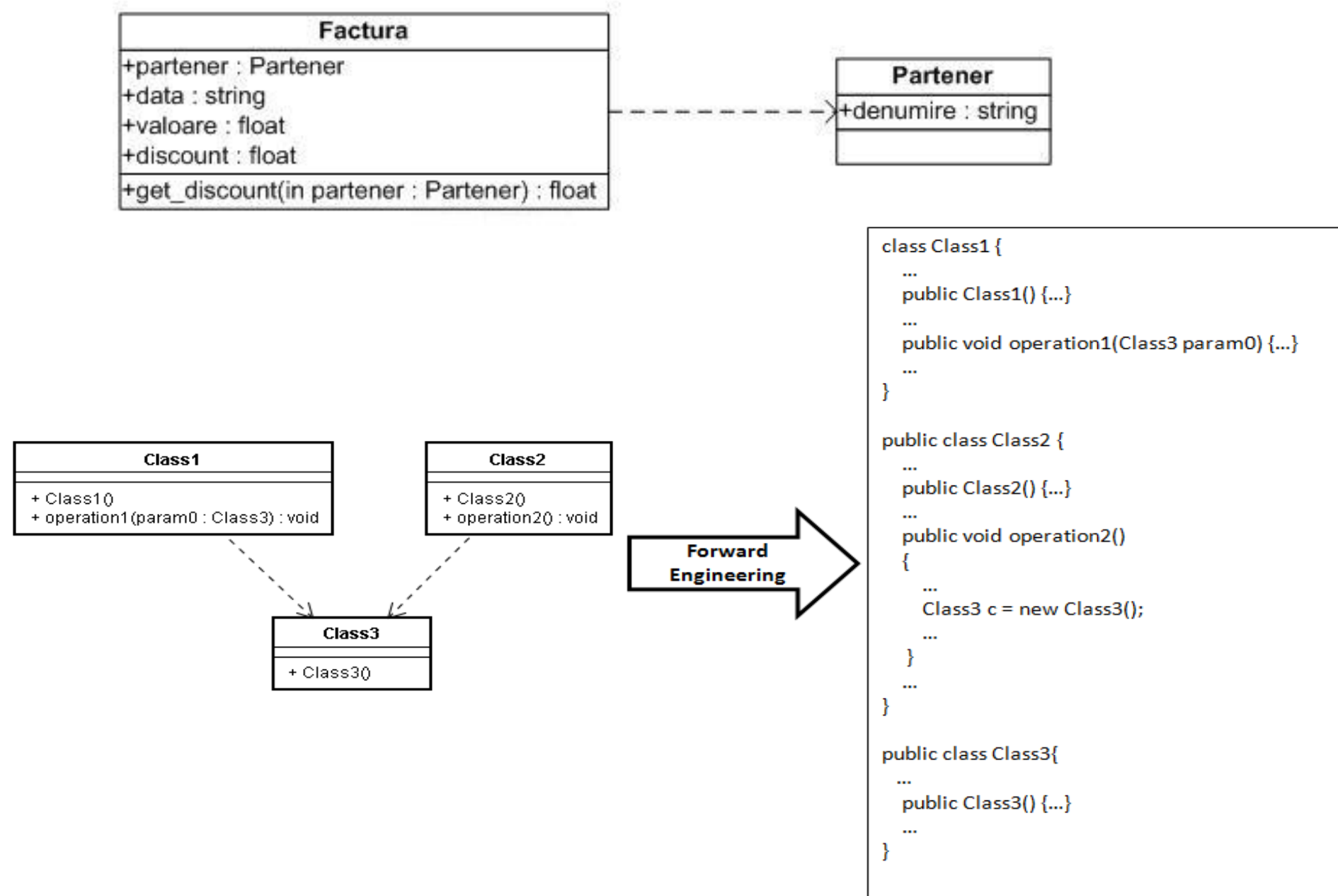
Forward engineering for association relationship (1)



Forward engineering for association relationship (2)

UML diagrams – Class Diagram

- Example of dependency - The verb used is "to use"



Forward engineering for the dependency relationship

UML diagrams – Class Diagram

- Example of aggregations
 - The **Catalog** may have several **Products** at the same time, a **Product** may exist even if that catalog does not exist

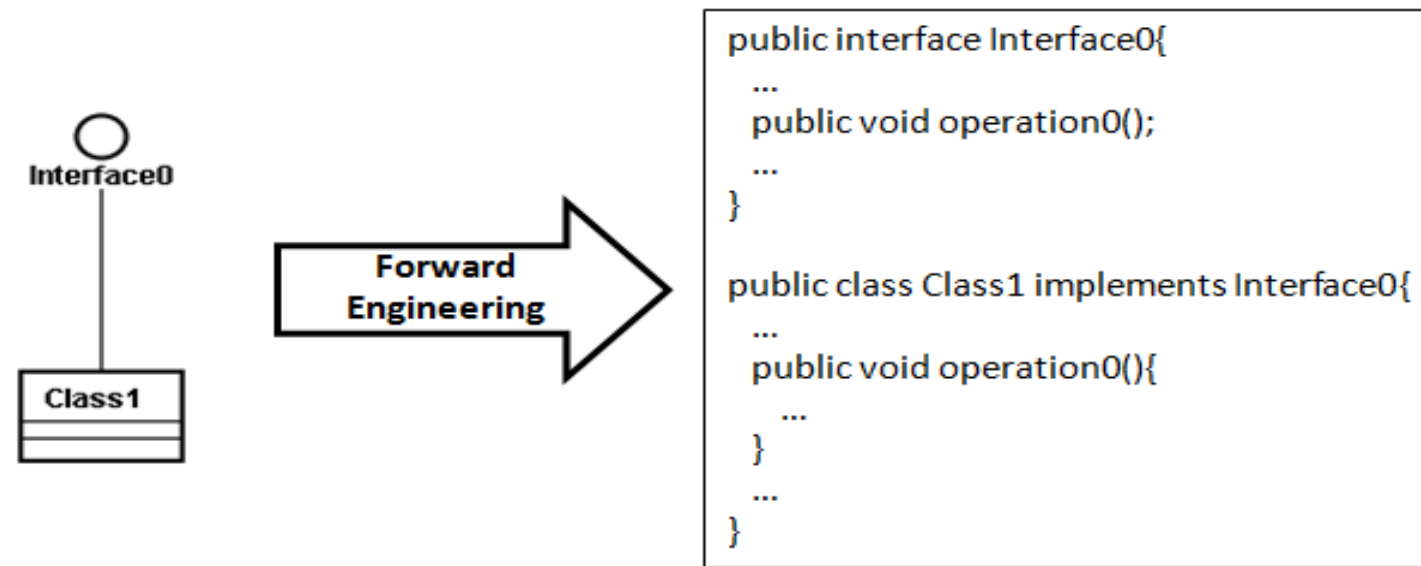


- Example of composition
 - The instance of the **Commission** class exists as long as there is a court of the **Exam** class



UML diagrams – Class Diagram

- Forward engineering for the realization relationship



UML diagrams - Use-case Diagram

- A case diagram is one of the diagrams used in UML to model the dynamic aspects of a program along with the activity diagram, state diagram, sequence diagram, and collaboration diagram
- The component of the use case diagrams
 - **Use case-uri**
 - **Actors**
 - **Relations** between use cases, between actors and between use cases and actors
- A use case represents the user's requirements
 - It is a description of a set of action sequences that a program executes when interacting with entities outside it (actors) and leads to an results that is observable and useful for the actor
 - A use case describes what a program / subprogram does, but does not specify anything about how a particular functionality is implemented

UML diagrams - Use-case Diagram

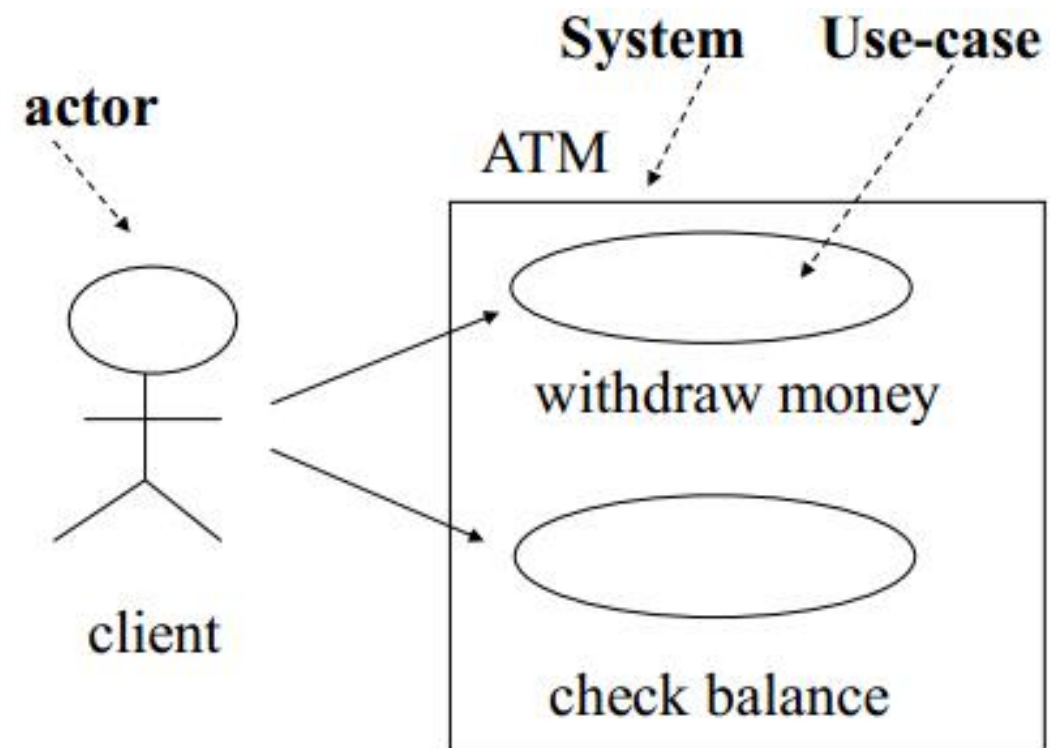
- Use case
 - Identifying the **use cases** is done starting from user requirements and analyzing the description of the problem
 - Every **use case** has a name that differs from other use cases.
 - The behavior of a use case can be specified by describing an flow of events in clear enough text to be understood by someone from outside (e.g. the user)
 - » These flows of events represents possible scenarios of using the system
- Actors
 - Is the idealization of a person, process, or external object that interacts with a system, subsystem, or class
 - Are entities outside the system
 - They can be users (people), hardware or other programs
 - Each actor has a name that indicates the role he plays in interacting with the program
 - Each actor must communicate with at least one use case
- Relations
 - Express interactions between use case, between actors and between use case and actors.

UML diagrams - Use-case Diagram

Element	Description	Notation
Actor	An actor is a user of the system, but also can be on other system that interacts with our system	
Use Case	Are represented as an eclipse with a name associated	
Association relation	Can be defined between actors and use cases, or between use cases. It is used to express the interaction (communication) between the elements it joins	

UML diagrams - Use-case Diagram

- Example of use case diagram for an ATM system



Use case: <use case goal>
Primary actor: <a role name for the actor who initiates the use case>
Main success scenario: <the steps of the main success scenario from trigger to goal delivery>
Extensions: <alternate scenarios of success or failure>

UML diagrams - Use-case Diagram

- **Use case title: withdraw money**
 - **Summary :This use case allows allows a card holder to withdraw money**
 - Actors: Card Holder
 - Preconditions: ATM has money
 - Main success scenario:
 1. Card Holder introduces the card in ATM
 2. The ATM checks if the card is valid
 3. The ATM requests the cardholder to insert the pin
 4. The cardholder introduces the pin
 5. The ATM checks if the date is correct
 6. The ATM requests the cardholder to specify the quantity of money
 7. The cardholder introduces the quantity of money
 8. The ATM checks if the quantity of money is bellow the daily limit
 9. The ATM ask the cardholder if he wants a receipt
 10. The cardholder specifies that he wants the receipt
 11. The ATM returns the card
 12. The cardholder take the card
 13. The ATM returns the money and the receipt
 14. The card holder takes the money and the receipt

UML diagrams - Use-case Diagram

- **Use case title: withdraw money**
 - **Summary :This use case allows allows a card holder to withdraw money**
 - Actors: Card Holder
 - Preconditions: ATM has money
 - Alternative sequences:
 - a) Incorrect pin
 1. The ATM tells the cardholder that the pin is incorrect
 2. The scenario returns to step 3
 - a) The quantity of money is higher that the daily limit
 1. The ATM tells the cardholder that quantity of money is higher that the daily limit
 2. The scenario returns to step 7

UML diagrams - Use-case Diagram

- **Use case title: withdraw money**
 - **Summary :**This use case allows allows a card holder to withdraw money
 - Actors: Card Holder
 - Preconditions: ATM has money
 - Errors sequences:
 - a) Invalid card: appears to step 2
 - 1. The ATM tells the cardholder that the card is invalid (e.g. the card is expired) and suppresses the card. In this case the use case fails.

Swing - API

- Is part of Java Foundation Classes (JFC)
 - Offers facilities to write applications with a graphical user interface functional and aesthetic enriched
- Includes 17 packages consisting of classes and interfaces
- **javax.swing** - Is the most important package from Swing
 - Contains the basic components to create graphical user interface
 - » Atomic components (e.g. **JLabel**, **JButton**, **JCheckBox**, **JRadioButton**, **JToggleButton**, **JScrollBar**, **JSlider**)
 - » Complex components (e.g. **JTable**, **JTree**, **JComboBox**, **JList**, **JFileChooser**, **JColorChooser**, **JOptionPane**)
 - » Components for editing text (e.g. **JTextField**, **JPasswordField**, **JTextArea**)
 - » Menus (e.g. **JMenuBar**, **JMenu**, **JPopupMenu**, **JMenuItem**, **JCheckboxMenuItem**, **JRadioButtonMenuItem**)
 - » Intermediate containers (e.g. **JPanel**, **JTabbedPane**, **JDesktopPane**)
 - » High level containers (e.g. **JFrame**, **JDialog**, **JApplet**)

Swing - Windows

- **JFrame** is a high level container that allows to display graphical components on the screen
- The body of the window is an instance of the **Container** class that can be obtained with **getContentPane** method

```
JFrame jf = new JFrame();  
jf.getContentPane().setLayout(new FlowLayout());  
jf.getContentPane().add(new JButton("OK"));
```

- The default behaviour of a **JFrame** object when we close the window is to hide the window when the user press the close button
- The default behaviour of a **JFrame** object can be modified with **setDefaultCloseOperation** method that receives as arguments constants from **WindowConstants** and **JFrame** classes

```
jf.setDefaultCloseOperation (WindowConstants.HIDE_ON_CLOSE);  
jf.setDefaultCloseOperation (WindowConstants.DO_NOTHING_ON_CLOSE);  
jf.setDefaultCloseOperation (JFrame.EXIT_ON_CLOSE);
```

Swing – Layout Manager

- Object that controls the size and position of the components within a container
- A layout manager is an object that implements the **LayoutManager** interface
- When a container is instantiate, a default layout manager associated to container is created
- Examples of Layout Manager
 - **FlowLayout** - arranges the components horizontally from left to right, one by one, on lines
 - **BorderLayout** - divides the display area in five regions: north, south, east, west, and center; when you add a new component, you need to specifiy the position

`panel.add(component, BorderLayout.NORTH)`
 - **GridLayout** -places components in a grid of cells (the number of rows and columns are fixed); each component takes all the available space within its cell, and each cell is exactly the same size
 - **CardLayout** -treats each component in the container as a card; only one card is visible at a time, and the container acts as a deck of cards
 - **GridBagLayout** -places components in a grid of rows and columns, allowing specified components to span multiple rows or columns

Swing –Event Handling (https://www.tutorialspoint.com/swing/swing_event_handling.htm)

- Events
 - Change in the state of an object is known as **Event**, i.e., event describes the change in the state of the source
 - Are generated as consequences of a person interacting with the graphical components in the GUI
 - » For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list
- What is Event Handling?
 - Event Handling is the mechanism that controls the event and decides what should happen if an event occurs
 - This mechanism has a code which is known as an event handler, that is executed when an event occurs

Swing –Event Handling

- Event Handling Example

```
package com.tutorialspoint.gui;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class SwingControlDemo {
    private JFrame mainFrame;
    private JLabel headerLabel;
    private JLabel statusLabel;
    private JPanel controlPanel;

    public SwingControlDemo(){
        prepareGUI();
    }

    public static void main(String[] args){
        SwingControlDemo swingControlDemo = new SwingControlDemo();
        swingControlDemo.showEventDemo();
    }

    private void prepareGUI(){
        mainFrame = new JFrame("Java SWING Examples");
        mainFrame.setSize(400,400);
        mainFrame.setLayout(new GridLayout(3, 1));

        headerLabel = new JLabel("",JLabel.CENTER );
        statusLabel = new JLabel("",JLabel.CENTER);
        statusLabel.setSize(350,100);

        mainFrame.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent windowEvent){
                System.exit(0);
            }
        });
        controlPanel = new JPanel();
        controlPanel.setLayout(new FlowLayout());

        mainFrame.add(headerLabel);
        mainFrame.add(controlPanel);
        mainFrame.add(statusLabel);
        mainFrame.setVisible(true);
    }
}
```

```
private void showEventDemo(){
    headerLabel.setText("Control in action: Button");

    JButton okButton = new JButton("OK");
    JButton submitButton = new JButton("Submit");
    JButton cancelButton = new JButton("Cancel");

    okButton.setActionCommand("OK");
    submitButton.setActionCommand("Submit");
    cancelButton.setActionCommand("Cancel");

    okButton.addActionListener(new ButtonClickListener());
    submitButton.addActionListener(new ButtonClickListener());
    cancelButton.addActionListener(new ButtonClickListener());

    controlPanel.add(okButton);
    controlPanel.add(submitButton);
    controlPanel.add(cancelButton);

    mainFrame.setVisible(true);
}

private class ButtonClickListener implements ActionListener{
    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();

        if( command.equals( "OK" ) ) {
            statusLabel.setText("Ok Button clicked.");
        } else if( command.equals( "Submit" ) ) {
            statusLabel.setText("Submit Button clicked.");
        } else {
            statusLabel.setText("Cancel Button clicked.");
        }
    }
}
}
```

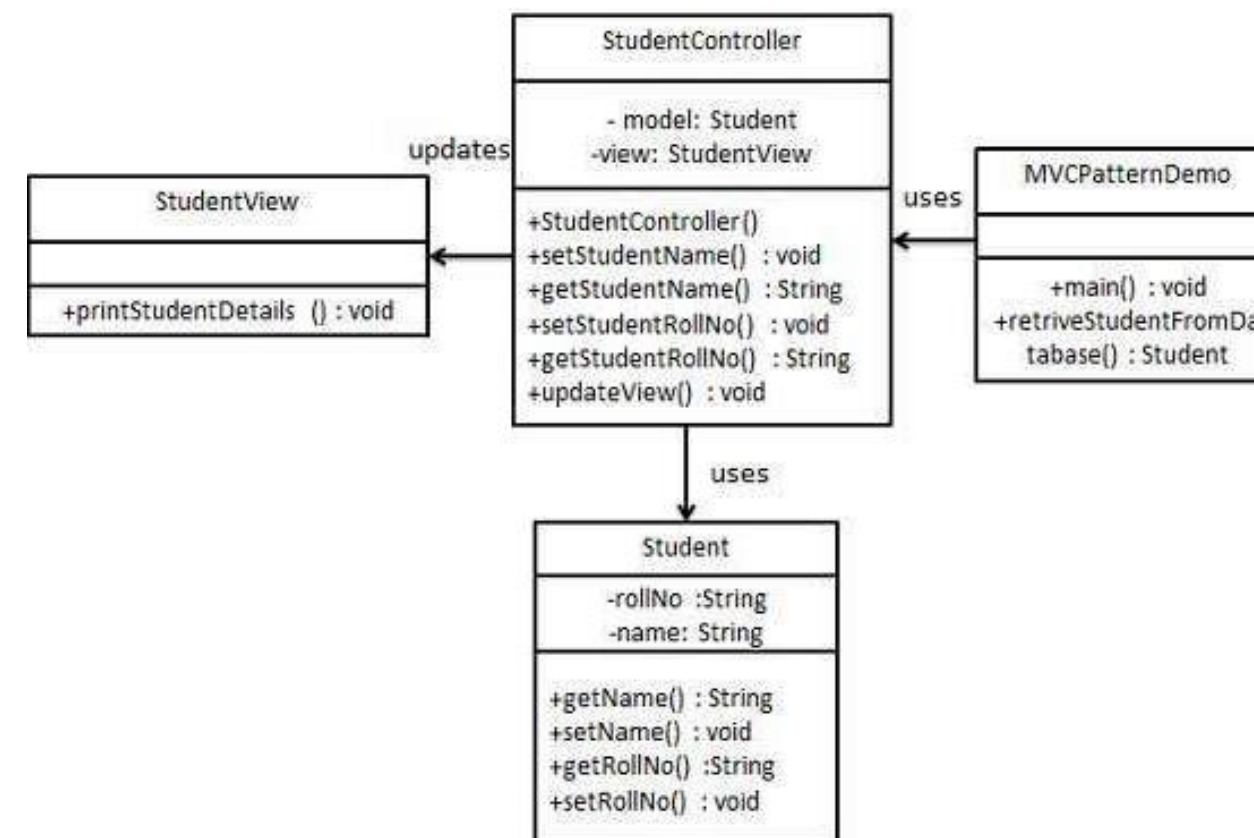


Model-View-Controller (MVC) Pattern

- Is an architectural pattern that divides an application into three interconnected part
 - **Model**
 - » Represent knowledge. A model could be a single object, or it could be some structure of objects
 - **View**
 - » Represents the visualization of the data that model contains
 - **Controller**
 - » Acts on both model and view
 - » Controls the data flow into model object and updates the view whenever data changes
 - » Keeps view and model separate

MVC Pattern – Example 1

- Example from: https://www.tutorialspoint.com/design_pattern/mvc_pattern.htm
- *Student* class acts as a model
- *StudentView* class will be a view class which can print student details on console
- *StudentController* class is the controller class responsible to store data in *Student* object and update view *StudentView* accordingly



MVC Pattern –Example 1

Step 1

Create Model.

Student.java

```
public class Student {
    private String rollNo;
    private String name;

    public String getRollNo() {
        return rollNo;
    }

    public void setRollNo(String rollNo) {
        this.rollNo = rollNo;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Step 5

Verify the output.

```
Student:
Name: Robert
Roll No: 10
Student:
Name: John
Roll No: 10
```

Step 2

Create View.

StudentView.java

```
public class StudentView {
    public void printStudentDetails(String studentName, String studentRollNo){
        System.out.println("Student: ");
        System.out.println("Name: " + studentName);
        System.out.println("Roll No: " + studentRollNo);
    }
}
```

Step 4

Use the *StudentController* methods to demonstrate MVC design pattern usage.

MVCPatternDemo.java

```
public class MVCPatternDemo {
    public static void main(String[] args) {

        //fetch student record based on his roll no from the database
        Student model = retrieveStudentFromDatabase();

        //Create a view : to write student details on console
        StudentView view = new StudentView();

        StudentController controller = new StudentController(model, view);

        controller.updateView();

        //update model data
        controller.setStudentName("John");

        controller.updateView();
    }

    private static Student retrieveStudentFromDatabase(){
        Student student = new Student();
        student.setName("Robert");
        student.setRollNo("10");
        return student;
    }
}
```

Step 3

Create Controller.

StudentController.java

```
public class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view){
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name){
        model.setName(name);
    }

    public String getStudentName(){
        return model.getName();
    }

    public void setStudentRollNo(String rollNo){
        model.setRollNo(rollNo);
    }

    public String getStudentRollNo(){
        return model.getRollNo();
    }

    public void updateView(){
        view.printStudentDetails(model.getName(), model.getRollNo());
    }
}
```

MVC Pattern - Example 2 from <https://www.lepoint.net/GUI/structure/40mvc.html>

```
// structure/calc-mvc/CalcModel.java
// Fred Swartz - December 2004
// Model
// This model is completely independent of the user interface.
// It could as easily be used by a command line or web interface.

import java.math.BigInteger;

public class CalcModel {
    //... Constants
    private static final String INITIAL_VALUE = "0";

    //... Member variable defining state of calculator.
    private BigInteger m_total; // The total current value state.

    //===== constructor
    /** Constructor */
    CalcModel() {
        reset();
    }

    //===== reset
    /** Reset to initial value. */
    public void reset() {
        m_total = new BigInteger(INITIAL_VALUE);
    }

    //===== multiplyBy
    /** Multiply current total by a number.
     * @param operand Number (as string) to multiply total by.
     */
    public void multiplyBy(String operand) {
        m_total = m_total.multiply(new BigInteger(operand));
    }

    //===== setValue
    /** Set the total value.
     * @param value New value that should be used for the calculator total.
     */
    public void setValue(String value) {
        m_total = new BigInteger(value);
    }

    //===== getValue
    /** Return current calculator total. */
    public String getValue() {
        return m_total.toString();
    }
}
```

```
// structure/calc-mvc/CalcView.java - View component
// Presentation only. No user actions.
// Fred Swartz -- December 2004

import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

class CalcView extends JFrame {
    //... Constants
    private static final String INITIAL_VALUE = "1";

    //... Components
    private JTextField m_userInputTf = new JTextField(5);
    private JTextField m_totalTf = new JTextField(20);
    private JButton m_multiplyBtn = new JButton("Multiply");
    private JButton m_clearBtn = new JButton("Clear");

    private CalcModel m_model;

    //===== constructor
    /** Constructor */
    CalcView(CalcModel model) {
        //... Set up the logic
        m_model = model;
        m_model.setValue(INITIAL_VALUE);

        //... Initialize components
        m_totalTf.setText(m_model.getValue());
        m_totalTf.setEditable(false);

        //... Layout the components.
        JPanel content = new JPanel();
        content.setLayout(new FlowLayout());
        content.add(new JLabel("Input"));
        content.add(m_userInputTf);
        content.add(m_multiplyBtn);
        content.add(new JLabel("Total"));
        content.add(m_totalTf);
        content.add(m_clearBtn);

        //... finalize layout
        this.setContentPane(content);
        this.pack();

        this.setTitle("Simple Calc - MVC");
        // The window closing event should probably be passed to the
        // Controller in a real program, but this is a short example.
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

```
void reset() {
    m_totalTf.setText(INITIAL_VALUE);
}

String getUserInput() {
    return m_userInputTf.getText();
}

void setTotal(String newTotal) {
    m_totalTf.setText(newTotal);
}

void showError(String errMessage) {
    JOptionPane.showMessageDialog(this, errMessage);
}

void addMultiplyListener(ActionListener mal) {
    m_multiplyBtn.addActionListener(mal);
}

void addClearListener(ActionListener cal) {
    m_clearBtn.addActionListener(cal);
}
}
```

MVC Pattern - Example 2

```
// structure/calc-mvc/CalcController.java - Controller
// Handles user interaction with listeners.
// Calls View and Model as needed.
// Fred Swartz -- December 2004

import java.awt.event.*;

public class CalcController {
    //... The Controller needs to interact with both the Model and View.
    private CalcModel m_model;
    private CalcView m_view;

    //===== constructor
    /** Constructor */
    CalcController(CalcModel model, CalcView view) {
        m_model = model;
        m_view = view;

        //... Add listeners to the view.
        view.addMultiplyListener(new MultiplyListener());
        view.addClearListener(new ClearListener());
    }

    //////////////////////////////////////// inner class MultiplyListener
    /** When a multiplication is requested.
     * 1. Get the user input number from the View.
     * 2. Call the model to multiply by this number.
     * 3. Get the result from the Model.
     * 4. Tell the View to display the result.
     * If there was an error, tell the View to display it.
     */
    class MultiplyListener implements ActionListener {
        public void actionPerformed(ActionEvent e) {
            String userInput = "";
            try {
                userInput = m_view.getUserInput();
                m_model.multiplyBy(userInput);
                m_view.setTotal(m_model.getValue());
            } catch (NumberFormatException nex) {
                m_view.showError("Bad input: '" + userInput + "'");
            }
        }
    }
}

////////////////////////////////////// inner class ClearListener
/** 1. Reset model.
 * 2. Reset View.
 */
class ClearListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        m_model.reset();
        m_view.reset();
    }
}

}
}
}
```

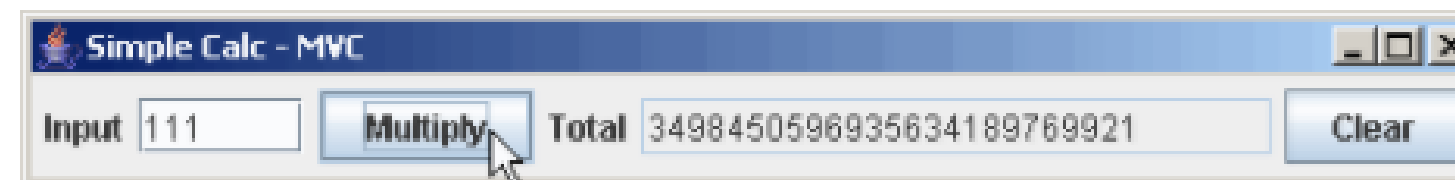
Main program

The main program initializes everything and ties everything together.

```

1 // structure/calc-mvc/CalcMVC.java -- Calculator in MVC pattern.
2 // Fred Swartz -- December 2004
3
4 import javax.swing.*;
5
6 public class CalcMVC {
7     //... Create model, view, and controller.  They are
8     //    created once here and passed to the parts that
9     //    need them so there is only one copy of each.
10    public static void main(String[] args) {
11
12        CalcModel    model    = new CalcModel();
13        CalcView      view     = new CalcView(model);
14        CalcController controller = new CalcController(model, view);
15
16        view.setVisible(true);
17    }
18 }

```



Unit Testing with JUnit

- Testing is the process of evaluating a system or its component(s) with the intent to find whether it satisfies the specified requirements or not
- Testing is executing a system in order to identify any gaps, errors, or missing requirements in contrary to the actual requirements
- JUnit is a **Regression Testing Framework** used by developers to implement unit testing in Java, and accelerate programming speed and increase the quality of code
- JUnit Framework can be easily integrated with either of the following –
 - Eclipse
 - Ant
 - Maven
- Where should the test be located?
 - **src/test/java**

Unit Testing with Junit

Configure Maven to work with Junit

Add the following dependencies to pom.xml

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
    <scope>test</scope>
    <exclusions>
      <exclusion>
        <groupId>org.hamcrest</groupId>
        <artifactId>hamcrest-core</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
  <!-- This will get hamcrest-core automatically -->
  <dependency>
    <groupId>org.hamcrest</groupId>
    <artifactId>hamcrest-library</artifactId>
    <version>1.3</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Unit Testing with JUnit – Features

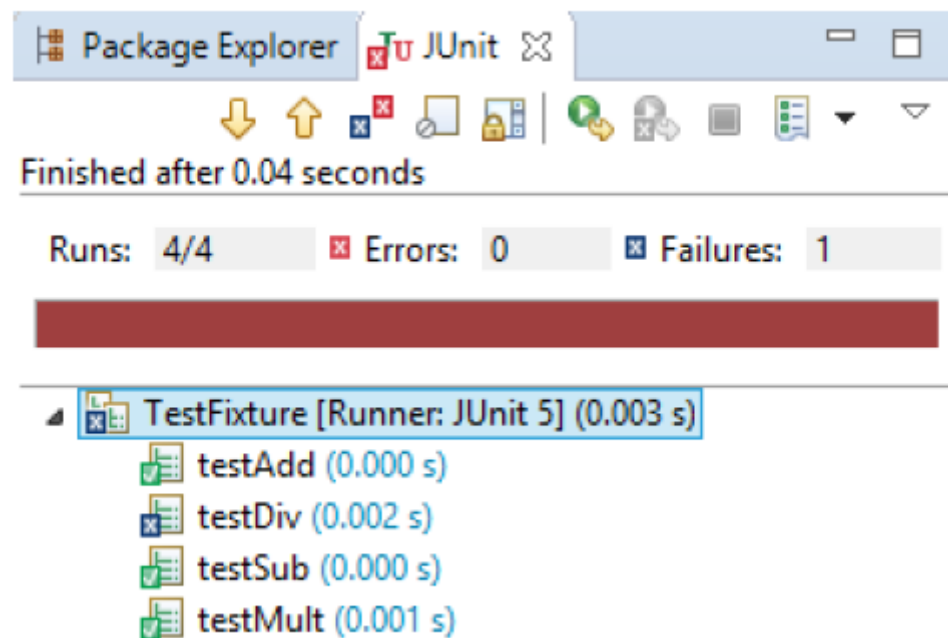
- JUnit test framework provides the following important features
 - Fixtures
 - Test suites
 - Test runners
 - JUnit classes

Unit Testing with Junit – Features

- Fixtures
 - A fixed state is a set of objects that prepared as a starting point for the tests
 - Running tests always start from this state
 - The method that are used are:
 - » setUp() method, which automatically runs before every test invocation
 - » tearDown() method, which automatically runs after every test method

Unit Testing with Junit – Features

- Fixtures - Example
 - When the example is run, we observed that the setUp() and tearDown() are run 4 times before and after the four methods: testAdd(), testSub(), testMult() and testDiv()
- The execution result in the case of **TestFixture** fails for testDiv() method:



```
import junit.framework.*;

public class TestFixture extends TestCase {
    public int n1, n2;

    // setarea valorilor n1, n2
    public void setUp() {
        n1=2;
        n2=3;

        System.out.println("setUp: n1: " + n1 + ", n2: " + n2);
    }

    // operatii cu cele doua valori
    public void testAdd() {
        double result= n1 + n2;
        assertTrue(result == 5);
    }

    public void testSub() {
        double result= n1 - n2;
        assertTrue(result < 0);
    }

    public void testMult() {
        double result= n1 * n2;
        assertTrue(result > 5);
    }

    public void testDiv() {
        double result= n1 / n2;
        assertTrue(result > 0);
    }

    //metoda rulata dupa executarea testelor
    public void tearDown() {
        n1 = 0;
        n2 = 0;
        System.out.println("tearDown: n1: " + n1 + ", n2: " + n2);
    }
}
```

Unit Testing with JUnit – Features

- Test Suites

- A test suite bundles a few unit test cases and runs them together.
- In JUnit, both `@RunWith` and `@Suite` annotation are used to run the suite test

```
import org.junit.Test;
import org.junit.Ignore;
import static org.junit.Assert.assertEquals;

public class TestJUnit1 {

    String message = "Robert";
    MessageUtil messageUtil = new MessageUtil(message);

    @Test
    public void testPrintMessage() {
        System.out.println("Inside testPrintMessage()");
        assertEquals(message, messageUtil.printMessage());
    }
}
```

```
import org.junit.Test;
import org.junit.Ignore;
import static org.junit.Assert.assertEquals;

public class TestJUnit2 {

    String message = "Robert";
    MessageUtil messageUtil = new MessageUtil(message);

    @Test
    public void testSalutationMessage() {
        System.out.println("Inside testSalutationMessage()");
        message = "Hi!" + "Robert";
        assertEquals(message, messageUtil.salutationMessage());
    }
}
```

```
import org.junit.runner.RunWith;
import org.junit.runners.Suite;

//JUnit Suite Test
@RunWith(Suite.class)

@Suite.SuiteClasses({
    TestJUnit1.class, TestJUnit2.class
})

public class JunitTestSuite {
}
```



Unit Testing with Junit – Features

- Test Runners
 - Are used for executing the test cases
 - In example bellow assumes the test class **TestJunit** already exists

```
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class TestRunner {
    public static void main(String[] args) {
        Result result = JUnitCore.runClasses(TestJunit.class);

        for (Failure failure : result.getFailures()) {
            System.out.println(failure.toString());
        }

        System.out.println(result.wasSuccessful());
    }
}
```

Unit Testing with JUnit – Features

- JUnit Classes
 - Are important classes, used in writing and testing JUnits
 - Some of the important classes are
 - » **Assert** – Contains a set of assert methods
 - » **TestResult** – Contains methods to collect the results of executing a test case

Unit Testing with Junit – Example

- Basic example to demonstrate the step-by-step process of using Junit
(https://www.tutorialspoint.com/junit/junit_basic_usage.htm)
- Step 1: Create a java class to be tested (e.g. MessageUtil.java)

```
/*  
 * This class prints the given message on console.  
 */  
  
public class MessageUtil {  
  
    private String message;  
  
    //Constructor  
    //@param message to be printed  
  
    public MessageUtil(String message){  
        this.message = message;  
    }  
  
    // prints the message  
    public String printMessage(){  
        System.out.println(message);  
        return message;  
    }  
}
```

Unit Testing with JUnit – Example

- Step 2: Create Test Case Class
 - Create a java test class, TestJUnit.java
 - Add a test method testPrintMessage() to your test class
 - Add an Annotation @Test to method testPrintMessage()
 - Implement the test condition and check the condition using assertEquals API of JUnit

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class TestJUnit {

    String message = "Hello World";
    MessageUtil messageUtil = new MessageUtil(message);

    @Test
    public void testPrintMessage() {
        assertEquals(message,messageUtil.printMessage());
    }
}
```

Unit Testing with JUnit – Example

- Step 3: Create Test Runner Class
 - Create a **TestRunner** java class
 - Use **runClasses** method of **JUnitCore** class of JUnit to run the test case of the above created test class
 - Get the result of test cases run in **Result** Object
 - Get failure(s) using the **getFailures()** method of **Result** object
 - Get Success result using the **wasSuccessful()** method of **Result** object

```
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class TestRunner {
    public static void main(String[] args) {
        Result result = JUnitCore.runClasses(TestJunit.class);

        for (Failure failure : result.getFailures()) {
            System.out.println(failure.toString());
        }

        System.out.println(result.wasSuccessful());
    }
}
```

Unit Testing with Junit – Example

- Step 4:
 - Compile the **MessageUtil**, **Test** case and **Test Runner** classes using javac
 - Now run the Test Runner, which will run the test case defined in the provided Test Case class
 - Verify the output

```
Hello World  
true
```

Unit Testing with Junit – Assert Class

- Assert class from **org.junit** package
 - Provides a set of assertion methods useful for writing tests
 - » **void assertEquals(primitive expected, primitive actual)** – Check if two primitive values are equal
 - » **void assertEquals(Object expected, Object actual)** – Check if two objects are equal
 - » **void assertSame(Object expected, Object actual)** – Check if two objects refer to the same memory address
 - » **void assertNotSame(Object expected, Object actual)** – Check if two objects don't refer to the same memory address
 - » **void assertFalse(boolean condition)** – Check if the condition is false
 - » **void assertTrue(boolean condition)** – Check if the condition is true
 - » **void assertNotNull(Object object)** – Check if the object isn't null
 - » **void assertNull(Object object)** – Check if the object is null

Unit Testing with Junit – Assert Class

- Example

```
import org.junit.Test;
import static org.junit.Assert.*;

public class TestJUnit1 {
    @Test
    public void testAdd() {
        //test data
        int num = 5;
        String temp = null;
        String str = "JUnit is working fine";

        //check for equality
        assertEquals("JUnit is working fine", str);

        //check for false condition
        assertFalse(num > 6);

        //check for not null value
        assertNotNull(str);
    }
}
```

```
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class TestRunner1 {
    public static void main(String[] args) {
        Result result = JUnitCore.runClasses(TestJUnit1.class);

        for (Failure failure : result.getFailures()) {
            System.out.println(failure.toString());
        }

        System.out.println(result.wasSuccessful());
    }
}
```

Unit Testing with Junit – Basic Annotations

• Basic Annotations

- **@BeforeClass** – Run once before any of the test methods in the class, public static void
- **@AfterClass** – Run once after all the tests in the class have been run, public static void
- **@Before** – Run before @Test, public void
- **@After** – Run after @Test, public void
- **@Test** – This is the test method to run, public void

```
import org.junit.*;

public class BasicAnnotationTest {

    // Run once, e.g. Database connection, connection pool
    @BeforeClass
    public static void runOnceBeforeClass() {
        System.out.println("@BeforeClass - runOnceBeforeClass");
    }

    // Run once, e.g close connection, cleanup
    @AfterClass
    public static void runOnceAfterClass() {
        System.out.println("@AfterClass - runOnceAfterClass");
    }

    // Should rename to @BeforeTestMethod
    // e.g. Creating an similar object and share for all @Test
    @Before
    public void runBeforeTestMethod() {
        System.out.println("@Before - runBeforeTestMethod");
    }

    // Should rename to @AfterTestMethod
    @After
    public void runAfterTestMethod() {
        System.out.println("@After - runAfterTestMethod");
    }

    @Test
    public void test_method_1() {
        System.out.println("@Test - test_method_1");
    }

    @Test
    public void test_method_2() {
        System.out.println("@Test - test_method_2");
    }

}
```

Output

```
@BeforeClass - runOnceBeforeClass

@Before - runBeforeTestMethod
@Test - test_method_1
@After - runAfterTestMethod

@Before - runBeforeTestMethod
@Test - test_method_2
@After - runAfterTestMethod

@AfterClass - runOnceAfterClass
```

Unit Testing with Junit - Parameterized Test

Passing the parameters into the unit test method using the constructor [2]

```
@RunWith(value = Parameterized.class)
public class ParameterizedTest {

    private int numberA;
    private int numberB;
    private int expected;

    // Inject via constructor
    // for {8, 2, 10}, numberA = 8, numberB = 2, expected = 10
    public ParameterizedTest(int numberA, int numberB, int expected) {
        this.numberA = numberA;
        this.numberB = numberB;
        this.expected = expected;
    }

    // name attribute is optional, provide a unique name for test
    // multiple parameters, uses Collection<Object[]>
    @Parameters(name = "{index}: testAdd({0}+{1}) = {2}")
    public static Collection<Object[]> data() {
        return Arrays.asList(new Object[][]{
            {1, 1, 2},
            {2, 2, 4},
            {8, 2, 10},
            {4, 5, 9},
            {5, 5, 10}
        });
    }

    @Test
    public void test_addTwoNumbes() {
        assertThat(MathUtils.add(numberA, numberB), is(expected));
    }
}
```

Passing the parameters into the unit test method using fields injection via @Parameter [2]

```
@RunWith(value = Parameterized.class)
public class ParameterizedTest {

    //default value = 0
    @Parameter(value = 0)
    public int numberA;

    @Parameter(value = 1)
    public int numberB;

    @Parameter(value = 2)
    public int expected;

    @Parameters(name = "{index}: testAdd({0}+{1}) = {2}")
    public static Collection<Object[]> data() {
        return Arrays.asList(new Object[][]{
            {1, 1, 2},
            {2, 2, 4},
            {8, 2, 10},
            {4, 5, 9},
            {5, 5, 10}
        });
    }

    @Test
    public void test_addTwoNumbes() {
        assertThat(MathUtils.add(numberA, numberB), is(expected));
    }
}
```

```
public class MathUtils {

    public static int add(int a, int b) {
        return a + b;
    }
}
```

Unit Testing with Junit – Testing a List

```
public class ListTest {  
  
    @Test  
    public void testAssertList() {  
  
        List<String> actual = Arrays.asList("a", "b", "c");  
        List<String> expected = Arrays.asList("a", "b", "c");  
  
        //All passed / true  
  
        //1. Test equal.  
        assertEquals(actual, expected);  
  
        //2. If List has this value?  
        assertTrue(actual, contains("b"));  
  
        //3. Check List Size  
        assertEquals(actual, size(3));  
  
        assertEquals(actual.size(), is(3));  
  
        //4. List order  
  
        // Ensure Correct order  
        assertEquals(actual, contains("a", "b", "c"));  
  
        // Can be any order  
        assertTrue(actual, containsInAnyOrder("c", "b", "a"));  
  
        //5. check empty list  
        assertTrue(actual, not(IsEmptyCollection.empty()));  
  
        assertTrue(new ArrayList<>(), IsEmptyCollection.empty());  
  
    }  
}
```